



Disciplina: Programação 3 - Programação Funcional

Professor: Emanuel Barreiros

Assunto: Criando tipos e classes

Resumo: Utilize a linguagem de programação Haskell para resolver os problemas desta lista. Esta lista aborda o conceito de criação de tipos e classes.

- 1) Semelhante à função **somar**, defina uma função de multiplicação recursiva para números naturais `mult :: Nat -> Nat -> Nat`. Dica: use a função `somar` definida no [material de aula](#) para os números naturais.

- 2) O Prelude define o tipo `Ordering`

```
data Ordering = LT | EQ | GT
```

e a função

```
compare :: Ord a => a -> a -> Ordering
```

que decide se o primeiro valor recebido como argumento é menor (LT), igual (EQ) ou maior (GT) que o segundo argumento. Usando essa função redefina a função `existe :: Ord a => a -> Arvore a -> Bool` para árvores binárias de busca.

- 3) Considere o seguinte tipo de árvores binárias:

```
data Arvore a = Folha a | No (Arvore a) (Arvore a)
```

Digamos que a árvore é balanceada se a quantidade de folhas do lado esquerdo e do lado direito de todos os nós são iguais ou sua diferença é no máximo 1, e suas folhas são consideradas balanceadas por definição. Defina uma função

`balanceada :: Arvore a -> Bool` que decide se uma árvore é balanceada ou não. **IMPORTANTE:** Uma árvore está balanceada se todas as suas sub-árvores também estão balanceadas. **DICA:** primeiro defina uma função que conta a quantidade de folhas em uma árvore.

- 4) Defina a função `balancear :: [a] -> Arvore a` que converte uma lista não vazia em uma árvore balanceada. Dica: primeiro defina uma função que divide uma lista em duas metades cujos tamanhos diferem em no máximo 1.

- 5) Dada a definição

```
data Expr = Val Int | Add Expr Expr
```

defina a função



```
avaliar :: Expr -> Int
```

tal que `avaliar` substitui cada construtor `Val` na expressão pelo valor `Int` representado pelo construtor, e cada construtor `Add` pela aplicação da função `(+)`.

6) Utilizando a definição

```
data Expr = Val Int | Op Expr Expr
```

defina a função de alta ordem

```
folde :: (Int -> a) -> (a -> a -> a) -> Expr -> a
```

tal que `folde f g` substitui cada construtor `Val` na expressão pela aplicação da função `f` ao valor representado pelo `Val`, e cada construtor `Op` pela aplicação da função `g` aos valores resultantes de ambas as expressões codificadas pelo construtor `Op`.

- 7) Usando a função `folde`, defina a função `eval :: Expr -> Int` que avalia uma expressão para um valor inteiro. Uma forma de enxergar o que a função `eval` deve fazer é refletir sobre a seguinte frase: como eu posso usar a função `folde` de forma que `eval` avalie a expressão assumindo que `Op` signifique a soma? Por exemplo, se eu executasse `eval (Op (Val 1) (Val 4))`, assumindo que `Op` é a soma, ela deveria retornar 5. Perceba que `Op` poderia representar qualquer operação sobre os valores, basta que você forneça a operação desejada para a função `folde`.